



佛山大学
FOSHAN UNIVERSITY



计算机与人工智能学院
School of Computer Science and Artificial Intelligence

Section 2

NLP—正则表达式

Natural Language Processing — Regular Expression

任课教师：陈荟慧



基本要求

- (1) 了解正则表达式概念。
- (2) 熟悉常用的正则表达式函数。
- (3) 熟悉正则表达式元字符技术。
- (4) 掌握正则表达式的应用。





国产大模型的最新新闻 (2025-03-06)

- 阿里凌晨开源新模型：性能对标DeepSeek-R1，**参数仅为其1/20**
- 3月6日凌晨3点，阿里巴巴发布并开源全新的推理模型**通义千问**QwQ-32B。阿里巴巴称，这是一款拥有320亿参数的模型，其性能可与具备6710亿参数（其中370亿被激活）的DeepSeek-R1媲美。
- 据悉，QwQ-32B 在一系列基准测试中进行了评估，测试了数学推理、编程能力和通用能力，并与OpenAI的o1-mini以及DeepSeek满血版及蒸馏版进行了比较。
- 在**测试数学能力的AIME24**评测集上，以及评估代码能力的LiveCodeBench 中，千问 QwQ-32B表现与DeepSeek-R1相当，远胜于o1-mini及相同尺寸的R1蒸馏模型。
- 在由Meta首席科学家杨立昆领衔的“最难LLMs评测榜” **LiveBench**、谷歌等提出的指令遵循能力**IFEval**评测集、由加州大学伯克利分校等提出的评估准确调用函数或工具方面的**BFCL**测试中，千问QwQ-32B的得分均超越了DeepSeek-R1。
- 官方介绍称，这一成果突显了**将强化学习应用于经过大规模预训练的强大基础模型的有效性**。此外，阿里团队还在推理模型中集成了与Agent相关的能力，使其能够在使用工具的同时进行批判性思考，并根据环境反馈调整推理过程。
- 除了性能上的大幅提升，**QwQ-32B的另外一个亮点是大幅降低了部署使用成本**，开发者和企业可以在消费级硬件上轻松将其部署到本地设备中。
- 资料显示，从2023年至今，阿里通义团队已开源200多款模型，包含**大语言模型千问Qwen**及**视觉生成模型万相Wan**等两大基模系列，覆盖从0.5B到110B等参数，实现了全模态、全尺寸大模型的开源。



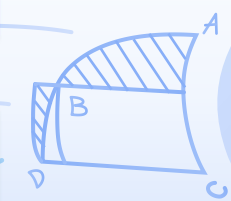
目录

1

正则表达式

2

任务：正则表达式应用



$$\Pi = 2X + a$$

$$A = \pi r^2$$



引导性提问

- (1) 正则表达式是什么?
- (2) 常用的正则表达式函数包括哪些?
- (3) 元字符是什么?
- (4) 有哪些字符是特殊字符?





正则表达式基础概念

- 定义：正则表达式是一种描述字符串模式的语法规则，用于对字符串进行匹配、查找、替换等操作。
- 作用：解决复杂文本处理需求，提高处理效率。如在日志分析中提取关键信息，验证用户输入格式是否正确。
- 例如，从一篇文章中找出所有英文单词，判断密码的安全等级





正则表达式函数

正则表达式是一种可以用于模式匹配和替换的工具。通过正则表达式可以对指定的文本实现匹配测试、内容查找、内容替换、字符串分割等功能。

正则表达式常用函数如下：

- **match函数**：用于检测字符串开头位置是否匹配。
- **search函数**：用于在整个字符串内查找对应的模式进行匹配。
- **findall函数**：返回的是正则表达式在字符串中所有匹配结果的列表。
- **sub函数**：能够找到所有模式匹配的字符串并用指定的字符串替换。





Python 中的 re 模块

- Python 通过内置的 re 模块支持正则表达式操作。
- 如 `re.search(r'hello', 'hello world')`。





正则表达式的元字符

元字符由特殊符号组成，元字符的应用是正则表达式强大的原因，**元字符定义了字符集合、子组匹配、模式重复次数**。元字符使得正则表达式不只是可以匹配一个字符串，还可以匹配字符串集合。

1. 字符匹配

- 英文句号 “.”：匹配任意一个字符，表示匹配除去换行符 “\n” 之外的任意一个字符。
- 中括号 “[]”：匹配多个字符，表示包含在中括号内部的字符都会被匹配。
- 管道符 “|”：用于对两个正则表达式进行或操作。
- 乘方符号 “^”：匹配字符串起始位置的内容。
- 货币符号 “\$”：匹配字符串的结束位置的内容。
- 量化符号 “?” “*” “+” “{n}” “{n,}” “{m,n}”：匹配需要的字符数。





正则表达式的元字符

- 量化符号的解释说明如表所示。
- 用 a^* 匹配零个或多个 a
- $a\{3\}$ 匹配三个 a

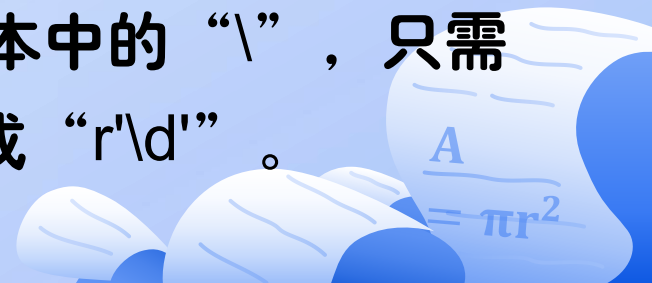
量化符	说明
$?$	前面的元素是可选，并且最多匹配1次
$*$	前面的元素会被匹配0次或多次
$+$	前面的元素会被匹配1次或多次
$\{n\}$	前面的元素会正好被匹配 n 次
$\{n, \}$	前面的元素至少会被匹配 n 次
$\{n$	前面的元素至少匹配 n 次，至多匹配 m 次





正则表达式的元字符

- 2. 转义字符 “\”
- 字符串中可以包含任何字符，如果待匹配的字符串中出现 “\$” “.” “[]” 等特殊字符，那么这将会与正则表达式的特殊字符发生冲突。
- 遇到这种情况，Python使用 “\” 将字符串内的特殊符号进行转义，即表示告诉Python，这个字符当作普通字符处理。
- “\” 是用于进行转义的，如果字符串包含 “\”，那么需要使用 “\” 将 “\” 进行转义。
- 在正则表达式中，通常解释一个反斜杠 “\” 需要用两个反斜杠 “\\” 表示。而python中自带的原生字符 “r” 很好的解决了这个问题。对于文本中的 “\”，只需要 “r\” 表示即可。对于需要转义的字符如 “\\d”，可以从写成 “r'\d'”。





正则表达式的元字符

- 例如，输入一个字符串 “C:\Users\Administrate\Desktop”，假设直接书写会直接报错。
- 报错原因是该字符串中带有转义符 “\”，第一个转义符后面带有U，则在读取过程中会被人为是 “\U”。
- 特殊字符中不带有这种字符，故而报错，后两个转义符亦是如此。
- 字符串应改为 “C:\\Users\\Administrate\\Desktop”。





正则表达式的元字符

Python预定义字符的解释说明如表所示。

预定义字符	说明
<code>\w</code>	字、字母、数字
<code>\W</code>	与 <code>\w</code> 反义，非数字和字母
<code>\s</code>	空白字符
<code>\S</code>	非空白字符
<code>\d</code>	数字
<code>\D</code>	非数字
<code>\b</code>	单词的边界
<code>\B</code>	非单词的边界





正则表达式举例

- 假设你有一个文本字符串 `text = "联系人信息：张三，电话：13800138000，邮箱：zhangsan@example.com；李四，电话：15612345678，地址：北京路1号"`，请使用 Python 语言中的 `re` 模块，编写正则表达式来提取其中所有的手机号码，并输出结果。输出格式：每行一个手机号码。
- **【输入样例】**
- 姓名：张三，电话号码：13709332123
- **【输出样例】**
- 13709332123
- **【输入样例】**
- 编号：012-12123456，电话号码：137 0933 2123
- **【输出样例】**
- 13709332123





提取手机号码正则表达式的解答步骤

- 1. 分析手机号码规则
- 中国手机号码由 11 位数字组成，第一位固定为 1，第二位数字取值范围为 3、5、6、7、8、9，后续 9 位为任意数字。
- 2. 构建正则表达式
- 起始数字 “1”：直接使用字符 “1”，因为手机号码第一位固定为 1。
- 第二位数字：由于第二位数字有特定取值范围，使用字符集合[356789]表示，方括号内列出所有可能的数字。
- 后续 9 位数字：使用\d表示任意数字，\d是预定义字符类，等价于[0-9]。要表示 9 个任意数字，使用量词{9}，即\d{9}。
- 将以上部分组合起来，完整的正则表达式为： $^1[356789]\d{9}$$ 。其中^和\$分别表示字符串的开头和结尾，确保匹配的是完整的 11 位手机号码，而不是字符串中包含的部分匹配。





提取手机号码正则表达式的解答步骤

- 3. 在 Python 中使用正则表达式提取手机号码
- `import re`
- `text = "联系人信息：张三，电话：13800138000，邮箱：zhangsan@example.com；李四，电话：15612345678，地址：北京路1号"`
- `pattern = r"1[356789]\d{9}"`
- `matches = re.findall(pattern, text)`
- `print(matches)`
- 使用`re.findall()`函数提取手机号码：`re.findall(pattern, text)`函数会在`text`字符串中查找所有符合`pattern`正则表达式的子字符串，并以列表形式返回。最后打印出提取到的手机号码列表。运行上述代码，将输出`['13800138000', '15612345678']`，成功提取出文本中的手机号码。





匹配模式与特殊字符 - 边界匹配

- **^与\$：** ^表示字符串开头，\$表示字符串结尾。例如^hello匹配以hello开头的字符串，world\$匹配以world结尾的字符串。
- **示例代码：** 展示验证输入字符串是否为指定格式开头或结尾的代码，如验证是否以数字开头 $^{[0 - 9]}$ 。





匹配模式与特殊字符 - 字符匹配

- 普通字符：普通字符匹配其自身，如a匹配字符a。
- 字符集合：使用方括号[]表示字符集合，匹配其中任意一个字符。例如[abc]匹配a、b或c；[a - z]匹配任意小写字母。
- 示例
 - [0 - 9]匹配任意数字
 - [A - Za - z]匹配任意字母





匹配模式与特殊字符 - 转义字符

- 转义字符\`\`：用于匹配具有特殊含义的字符本身，或表示特殊字符类。
- 如\`\.`匹配点号，\`\d`匹配任意数字（等价于`[0 - 9]`），\`\w`匹配字母、数字或下划线（等价于`[A - Za - z0 - 9_]`）。
- 代码`\d{3}-\d{4}`可以匹配如`123 - 4567`格式字符串。





元字符

- 1. 字符匹配类元字符
 - - `\.`：匹配除换行符 (`\n`) 之外的任意单个字符。例如，`a.c` 可以匹配 `abc`、`a0c` 等，但不能匹配 `a\nc`。
 - - `[]`：字符集，匹配方括号内的任意一个字符。例如，`[abc]` 匹配 `a`、`b` 或 `c`；`[a-z]` 匹配任意小写字母；`[0-9]` 匹配任意数字；`[a-zA-Z0-9]` 匹配任意字母或数字。
 - - `^[^]`：反向字符集，匹配不在方括号内的任意一个字符。例如，`^[abc]` 匹配除 `a`、`b`、`c` 之外的任意字符。





元字符

• 2. 边界匹配类元字符

- - `^`：匹配字符串的开头。例如，`^hello` 表示匹配以 `hello` 开头的字符串。
- - `\$`：匹配字符串的结尾。例如，`world\$` 表示匹配以 `world` 结尾的字符串。
- - `\b`：匹配单词边界，即单词和非单词字符之间的位置。例如，`\bhello\b` 只匹配完整的单词 `hello`，而不匹配 `hellow` 中的 `hello`。
- - `\B`：匹配非单词边界，与 `\b` 相反。例如，`\Bhello\B` 可以匹配 `hellow` 中的 `hello`，但不匹配独立的单词 `hello`。





元字符

• 3. 重复匹配类元字符

- - `\*`: 匹配前面的字符或子表达式零次或多次。例如, `a\*` 可以匹配空字符串、`a`、`aa`、`aaa` 等。
- - `+`: 匹配前面的字符或子表达式一次或多次。例如, `a+` 可以匹配 `a`、`aa`、`aaa` 等,但不能匹配空字符串。
- - `?`: 匹配前面的字符或子表达式零次或一次。例如, `colou?r` 可以匹配 `color` 或 `colour`。
- - `{n}`: 恰好匹配前面的字符或子表达式 `n` 次。例如, `a{3}` 匹配 `aaa`。
- - `{n,}`: 至少匹配前面的字符或子表达式 `n` 次。例如, `a{3,}` 匹配 `aaa`、`aaaa` 等。
- - `{n,m}`: 匹配前面的字符或子表达式 `n` 到 `m` 次 (包含 `n` 和 `m`)。例如, `a{2,4}` 匹配 `aa`、`aaa`、`aaaa`。





元字符

- 4. 分组与引用类元字符
 - - `()`：分组，用于将多个字符或子表达式组合为一个单元，可以对这个单元应用重复匹配等操作，也用于提取匹配的子字符串。例如，`(ab)+` 表示 `ab` 重复一次或多次；`(\d{3})-(\d{4})` 可以匹配如 `123-4567` 这样的字符串，并可以分别提取出 `123` 和 `4567`。
 -





元字符

- 5. 特殊字符类元字符
 - `\`：转义字符，用于转义元字符，使其表示字符本身；也用于表示一些特殊字符类。例如，`\.` 匹配点号 `.`；`\d` 匹配任意数字（等价于 `[0-9]`）；`\w` 匹配字母、数字或下划线（等价于 `[a-zA-Z0-9_]`）；`\s` 匹配空白字符（包括空格、制表符、换行符等）；`\D` 匹配任意非数字字符（等价于 `[^0-9]`）；`\W` 匹配任意非单词字符（等价于 `[^a-zA-Z0-9_]`）；`\S` 匹配任意非空白字符（等价于 `[^ \t\n\r\f\v]`）。
 - `|`：逻辑或操作符，匹配 `|` 两边的任意一个表达式。例如，`cat|dog` 可以匹配 `cat` 或 `dog`。



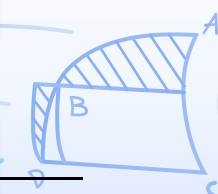
目录

1

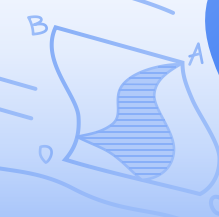
正则表达式

2

任务：正则表达式应用



$$\Pi = 2X + a$$



$$A = \pi r^2$$



《西游记》字符过滤

- 在进行中文分词前，数据格式要求全部是中文，此时需要对文本的以下内容进行过滤处理。
 - 特殊符号
 - 标点
 - 英文
 - 数字
- 读者也可以根据自己的要求过滤自定义字符。





自动提取人名与电话号码

- 文本人名正则表达式匹配方法如下。
 - 名字首先匹配第一个大写字母采用[A-Z]表示。
 - 名字第二个字符只包含英文句号、空格或字母，统一采用[\.a-zA-Z]表示。
 - 最后用+表示允许匹配多个。
- 电话号码正则表达式匹配方法如下。
 - 以 “ (” 或数字开头，采用[0-9(]表示。
 - 第二个字符接数字或 “) ” 或英文句号，采用[0-9-).]表示。
 - 最后用+表示允许匹配多个。





提取网页标签信息

- 例如，一个网页标签内容为 “百度 谷歌”。
- 需要提取网页标签中的网址和文本。
- 通过观察，网址信息保存在href属性中，文本内容则是在特殊字符中间。





正则表达式的应用场景

- **文本验证：**验证邮箱地址、身份证号码、电话号码等格式是否正确。展示邮箱地址正则表达式 $^[a-zA-Z0-9_+-.]@ [a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\$$ 及验证代码。
- **信息提取：**从日志文件、网页源码中提取关键信息。如从日志中提取时间、错误信息。
- **文本替换：**批量替换文本中符合特定模式的内容。例如将文本中所有数字替换为特定字符串。





案例 - 替换敏感词

- 案例描述：假设文本中包含敏感词，编写程序用正则表达式将敏感词替换为***。
- 敏感词处理思路：将敏感词组成字符集合，利用re.sub()函数进行替换。





小结

本章主要介绍正则表达式的基本知识和技术。

- 正则表达式的常用函数。
- 正则表达式的元字符及元字符的含义。
- 最后通过实例对正则表达式的使用方法进行展示。


$$\frac{A}{r} = \pi r^2$$